Chương 3 (Phần 2): Phát hiện lỗ hổng phần mềm – Phân tích tĩnh

3.5 Bounded Model Checking – 3.6 Mô hình bộ nhớ – 3.7 Aligned Memory Model

ThS. Vũ Quang Dũng

Khoa Hệ thống thông tin – Trường Đại học Phenikaa

CSE703093 – An toàn phần mềm

Nhắc lại Phần 1

- 3.1–3.3: Nền tảng verification/validation, soundness/completeness, static/dynamic/debug
- 3.4: SAT/SMT solving – DPLL, CDCL, DPLL(T), bit-vector/array theory, Z3

Phần 2 sẽ **kết hợp** Kripke structure (Chương 1) với SAT/SMT (mục 3.4) thành **Bounded Model Checking** – kỹ thuật kiểm chứng thực tế mạnh mẽ nhất trong chương này.

Nội dung Phần 2

1. 3.5 Kiểm chứng mô hình phần mềm (Bounded Model Checking)
2. 3.6 Mô hình bộ nhớ (Memory Model)
3. 3.7 Sắp xếp mô hình bộ nhớ (Aligned Memory Model)

Model Checking đầy đủ – Nhắc lại và giới hạn

- Model checking đầy đủ (Chương 1) duyệt *toàn bộ* không gian trạng thái của Kripke structure để kiểm tra $K \models \varphi$
- Giới hạn: **state explosion** – với chương trình thực tế, không gian trạng thái có thể là vô hạn hoặc quá lớn để duyệt hết
- **Bounded Model Checking (BMC)**: thay vì duyệt toàn bộ, chỉ kiểm tra các đường thực thi có độ dài **tối đa** k **bước** – đánh đổi độ đầy đủ lấy khả năng mở rộng

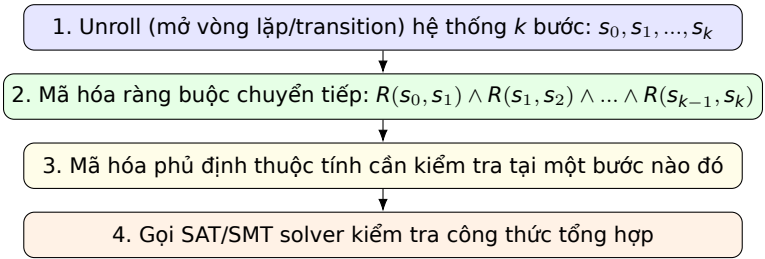
Ý tưởng cốt lõi của BMC

Định nghĩa

Bounded Model Checking: kiểm tra xem có tồn tại một đường thực thi *độ dài tối đa* k từ trạng thái khởi tạo vi phạm thuộc tính φ hay không, bằng cách mã hóa bài toán thành một công thức SAT/SMT và gọi solver.

Nếu SAT: solver trả về counterexample cụ thể (chính là phép gán thỏa mãn). Nếu UNSAT: **không có** vi phạm trong phạm vi k bước (nhưng chưa chắc đúng ngoài phạm vi này!).

Quy trình mã hóa BMC thành SAT/SMT



Công thức BMC tổng quát

Để kiểm tra thuộc tính safety **AG** $\neg$bad (không bao giờ đạt trạng thái xấu) trong phạm vi k bước:

$$\text{BMC}_k = I(s_0) \wedge \bigwedge_{i=0}^{k-1} R(s_i, s_{i+1}) \wedge \bigvee_{i=0}^{k} \text{bad}(s_i)$$

trong đó $I(s_0)$ là ràng buộc trạng thái khởi tạo. Nếu công thức này **SAT**, tồn tại một đường đi (counterexample) vi phạm thuộc tính trong $\leq k$ bước.

Ví dụ minh họa: Unroll mô hình đăng nhập (Chương 1)

Nhắc lại mô hình `ch1_models.kripke.build_login_example()`: $S = \{\text{idle, authenticated, locked}\}$.
Kiểm tra thuộc tính: ”Không bao giờ đạt `locked` trong 2 bước đầu” – với $k = 2$:

$$I(\text{idle}) \wedge R(s_0, s_1) \wedge R(s_1, s_2) \wedge (\text{locked}(s_0) \vee \text{locked}(s_1) \vee \text{locked}(s_2))$$

Solver sẽ tìm $s_0 =$idle, $s_1 =$locked $\Rightarrow$ SAT, counterexample: idle $\rightarrow$ locked ngay bước đầu tiên – thuộc tính **bị vi phạm**.

Cài đặt BMC đơn giản bằng Z3 (Python)

```python
from z3 import Solver, Bool, Or, Not, sat

def bmc_check(k, initial, transition, bad):
    s = Solver()
    states = [f"s{i}" for i in range(k + 1)]
    # ... them rang buoc initial(s0), transition(s_i, s_i+1) ...
    s.add(Or([bad(state) for state in states]))
    return s.check() == sat   # True = tim thay vi pham trong k buoc
```

Đây là khung sườn ý tưởng; module Python `ch3_staticsat` sẽ cài đặt đầy đủ và cụ thể hơn.

Completeness Threshold – Khi nào BMC trở thành đầy đủ

Định nghĩa

Completeness Threshold (CT): giá trị k đủ lớn sao cho nếu không tìm thấy vi phạm trong $\leq CT$ bước, thì **chắc chắn không có vi phạm nào** ở bất kỳ độ dài nào (đạt được đầy đủ - complete).

Với hệ thống hữu hạn trạng thái, CT thường liên quan tới *đường kính* (diameter) của đồ thị trạng thái – số bước tối đa cần để đạt mọi trạng thái reachable từ trạng thái khởi tạo.

BMC không đạt Completeness Threshold: Rủi ro

Lưu ý

Nếu chỉ chạy BMC với k nhỏ (chưa đạt CT) và không tìm thấy vi phạm, điều đó **không chứng minh** chương trình an toàn – vi phạm có thể tồn tại ở bước $k+1, k+2, \ldots$ Đây chính là lý do BMC mặc định là kỹ thuật **under-approximation** (đã học ở mục 3.2): tìm lỗi đáng tin cậy, nhưng không chứng minh vắng mặt lỗi trừ khi đạt CT.

CBMC – Bounded Model Checker công nghiệp cho C

- **CBMC** (C Bounded Model Checker): công cụ BMC thực tế phổ biến nhất cho mã C/C++
- Tự động: đọc mã nguồn C $\rightarrow$ unroll vòng lặp k lần $\rightarrow$ mã hóa thành SAT/SMT $\rightarrow$ gọi solver (thường là MiniSat hoặc Z3 nội bộ)
- Kiểm tra tự động: buffer overflow, division by zero, integer overflow, assertion failures, và nhiều thuộc tính người dùng tự định nghĩa

Sử dụng CBMC – Ví dụ cơ bản

```
cbmc vulnerable.c --bounds-check --pointer-check --div-by-zero-check
```

- `--bounds-check`: kiểm tra truy cập mảng ngoài phạm vi
- `--pointer-check`: kiểm tra dereference con trỏ NULL/dangling
- `--div-by-zero-check`: kiểm tra chia cho 0

Ví dụ CBMC phát hiện Buffer Overflow

```
void copy_input(char *user_input, int len) {
    char buffer[64];
    for (int i = 0; i < len; i++) {
        buffer[i] = user_input[i];  // CBMC se kiem tra: i < 64?
    }
}
```

```
$ cbmc buffer.c --bounds-check --unwind 100
FAILURE: buffer.c:4 array 'buffer' upper bound
  Counterexample: len = 65
```

CBMC trả về **giá trị cụ thể** (`len = 65`) gây vi phạm – đây chính là test case tự động sinh ra tương đương symbolic execution (Chương 1).

Loop Unwinding trong CBMC

- CBMC phải **unroll** (mở) mọi vòng lặp một số lần cố định k (tham số `--unwind`)
- Nếu vòng lặp thực tế chạy nhiều hơn k lần, CBMC sẽ chèn một *unwinding assertion* – cảnh báo nếu chương trình ”cố” chạy vượt quá giới hạn unwind đã chọn
- Chọn k quá nhỏ: có thể bỏ sót lỗi (không đạt completeness threshold); chọn k quá lớn: bài toán SAT trở nên rất lớn, chậm hoặc timeout

Unwinding Assertion – Ví dụ

```
$ cbmc loop.c --unwind 10
FAILURE: loop.c:5 unwinding assertion loop 0
```

Cảnh báo này nghĩa là: với `--unwind 10`, CBMC không thể xác nhận đã bao phủ *mọi* lần lặp có thể của vòng lặp – cần tăng giá trị unwind (hoặc dùng `--unwinding-assertions` kèm phân tích thêm) để có kết quả tin cậy hơn.

Bài tập: Xác định giá trị Unwind phù hợp

```c
void process(int n) {
    int arr[10];
    for (int i = 0; i < n; i++) {
        arr[i] = i;   // Loi neu n > 10
    }
}
```

Nếu muốn CBMC phát hiện được lỗi ngay cả khi n lên tới 15, giá trị `--unwind` tối thiểu nên là bao nhiêu?

Đáp án: Xác định giá trị Unwind phù hợp

- Cần `--unwind 15` (hoặc nhiều hơn) để CBMC ”mở” đủ số lần lặp bao phủ trường hợp $n = 15$
- Với `--unwind` nhỏ hơn 15 (vd: 10), CBMC sẽ báo unwinding assertion failure hoặc bỏ sót trường hợp $n > 10$ tùy cấu hình
- Thực hành tốt: chọn unwind bound lớn hơn giá trị lớn nhất *có ý nghĩa* của biến điều khiển vòng lặp, dựa trên hiểu biết miền giá trị thực tế

BMC cho thuộc tính bộ nhớ: Use-After-Free

```
void f(int flag) {
    int *p = malloc(sizeof(int));
    if (flag) free(p);
    if (flag) *p = 5;   // CBMC: use-after-free neu flag != 0
}
```

CBMC theo dõi trạng thái ”đã free” của mỗi con trỏ qua các bước unroll, mã hóa thành ràng buộc SMT (sử dụng array theory đã học ở mục 3.4) để tự động phát hiện use-after-free mà không cần chạy chương trình thực tế.

So sánh BMC với các kỹ thuật đã học

	Sound?	**Complete?**
Testing (Ch.1)	Không	Không
Static Analysis pattern-based (Ch.2)	Không	Không
BMC (dưới CT)	Có (trong phạm vi k)	Không (ngoài phạm vi k)
BMC (đạt CT)	Có	Có

BMC là kỹ thuật hiếm hoi có thể đạt *cả hai* tính chất khi đạt được completeness threshold – nhưng điều này không phải lúc nào cũng khả thi trong thời gian hợp lý.

Bài tập: Mã hóa BMC bằng tay

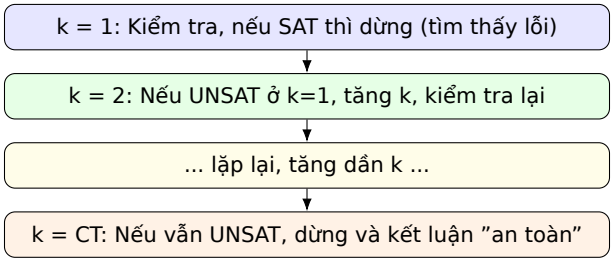
Hệ thống có 2 trạng thái $s_0 = 0, s_1 = 1$ (bit đơn giản), transition: $R(0,1)$ và $R(1,1)$ (self-loop). Thuộc tính xấu: ”đạt trạng thái 1”. Hãy viết công thức BMC cho $k = 1$ và xác định SAT/UNSAT.

Đáp án: Mã hóa BMC bằng tay

$$\text{BMC}_1 = (s_0 = 0) \wedge R(s_0, s_1) \wedge (\text{bad}(s_0) \vee \text{bad}(s_1))$$

Với $s_0 = 0$ (không bad), $R(0, 1)$ cho $s_1 = 1$ (bad) $\Rightarrow$ công thức **SAT** với $s_1 = 1$ – tìm thấy vi phạm ngay ở bước 1: hệ thống đạt trạng thái xấu chỉ sau 1 bước.

Incremental BMC – Tăng dần giá trị k



Chiến lược thực tế: tăng dần *k* thay vì chọn một giá trị cố định lớn ngay từ đầu, giúp tìm lỗi nông (shallow bug) nhanh hơn nhiều.

BMC cho hệ thống đa luồng (giới thiệu)

- Với chương trình đa luồng, BMC cần xét thêm *mọi thứ tự xen kẽ có thể* (interleaving) giữa các luồng
- Số lượng interleaving tăng theo cấp số nhân với số luồng và số bước – một dạng state explosion khác (đã học ở Chương 1)
- Kỹ thuật **Partial Order Reduction** giúp giảm số interleaving cần xét bằng cách loại bỏ các thứ tự tương đương nhau (cho cùng kết quả)

Ví dụ BMC phát hiện Race Condition

```
int balance = 100;
void withdraw(int amount) {
    if (balance >= amount) {     // Check
        balance -= amount;        // Use - co the co race o day!
    }
}
```

BMC đa luồng mã hóa *mọi* thứ tự xen kẽ có thể của 2 lời gọi `withdraw()` đồng thời, tìm ra tổ hợp khiến `balance` âm dù mỗi lần gọi riêng lẻ đều đúng logic – đây là race condition kinh điển (liên hệ TOCTOU đã học ở Chương 1).

Bài tập: BMC với vòng lặp

```c
int sum_array(int *arr, int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {
        total += arr[i];
    }
    return total;
}
```

Nếu muốn BMC kiểm chứng hàm này với mọi n từ 0 đến 50, giá trị `--unwind` tối thiểu là bao nhiêu? Điều gì xảy ra nếu chọn `--unwind 30`?

Đáp án: BMC với vòng lặp

- Cần `--unwind 50` (hoặc 51 để an toàn) để bao phủ trường hợp n=50
- Nếu chọn `--unwind 30`: với n từ 31-50, vòng lặp sẽ bị ”cắt” giữa chừng, CBMC sẽ báo *unwinding assertion failure* thay vì xác nhận an toàn – kết quả **không đáng tin cậy** cho các giá trị n lớn hơn 30

Bounded Model Checking vs. K-Induction

- BMC chỉ chứng minh ”an toàn tới bước k” – không tự động suy ra ”an toàn mãi mãi” trừ khi đạt CT
- **K-Induction**: kỹ thuật bổ sung chứng minh bằng quy nạp – nếu thuộc tính đúng ở k bước liên tiếp bất kỳ (base case) và việc nó đúng ở k bước kéo theo nó đúng ở bước $k+1$ (inductive step), thì đúng *với mọi* số bước
- K-Induction có thể chứng minh an toàn *vô hạn* mà không cần tính completeness threshold tường minh – kỹ thuật nâng cao dùng trong nhiều công cụ BMC hiện đại (bao gồm CBMC)

Tổng kết 3.5

- BMC: kiểm tra vi phạm trong phạm vi k bước bằng cách mã hóa thành SAT/SMT
- Sound trong phạm vi k, chỉ Complete khi đạt Completeness Threshold
- CBMC là công cụ công nghiệp phổ biến cho C, dùng loop unwinding và kiểm tra tự động nhiều loại lỗi bộ nhớ
- Chọn giá trị unwind/k phù hợp là đánh đổi giữa độ tin cậy và hiệu năng

Vì sao cần Memory Model hình thức?

- Để BMC/SMT có thể kiểm tra lỗi bộ nhớ (Chương 2), cần một cách biểu diễn **hình thức, chính xác** về cách bộ nhớ hoạt động
- Memory Model định nghĩa: bộ nhớ được biểu diễn thế nào, con trỏ nghĩa là gì, thao tác đọc/ghi/cấp phát/giải phóng được mã hóa ra sao trong logic

Mô hình bộ nhớ đơn giản nhất: Mảng phẳng (Flat Array)

```
from z3 import Array, BitVecSort, Solver

# Bo nho: anh xa dia chi (bit-vector 64-bit) toi gia tri byte (8-bit)
memory = Array('memory', BitVecSort(64), BitVecSort(8))
```

Mô hình đơn giản nhất: toàn bộ bộ nhớ là *một mảng lớn* ánh xạ địa chỉ tới giá trị byte, dùng chính **Array Theory** đã học ở mục 3.4 (`Store`/`Select`).

Mô hình hóa malloc và free hình thức

- `malloc(size)`: chọn một địa chỉ a "mới" (chưa cấp phát), đánh dấu vùng $[a, a + size)$ là *valid*
- `free(p)`: đánh dấu vùng nhớ tại p là *invalid* (không hợp lệ để truy cập)
- Mọi thao tác đọc/ghi cần kiểm tra: địa chỉ truy cập có nằm trong vùng *valid* hiện tại hay không

Mô hình hóa Valid Region bằng SMT

```python
from z3 import Array, BitVecSort, BoolSort, Solver, Not, Select

# valid_map: dia chi -> con hop le de truy cap hay khong (dung sau free)
valid_map = Array('valid', BitVecSort(64), BoolSort())

def check_use_after_free(addr):
    s = Solver()
    s.add(Not(Select(valid_map, addr)))  # addr KHONG hop le
    return s.check()   # Neu sat: co the xay ra truy cap vao vung da free
```

Đây chính là nguyên lý CBMC dùng nội bộ để phát hiện use-after-free đã thấy ở mục 3.5.

Points-to Analysis (giới thiệu)

Định nghĩa

Points-to Analysis: kỹ thuật static analysis xác định (một cách xấp xỉ) tập các vùng nhớ mà một con trỏ *có thể* trỏ tới tại một điểm trong chương trình.

Ví dụ

`int *p = cond ? &x : &y;` – points-to analysis xác định p có thể trỏ tới x *hoặc* y (tập $\{x, y\}$), không biết chính xác cái nào tại compile-time.

Đây là bài toán *khó* (đã đề cập ở Chương 2 mục 2.5) – độ chính xác của points-to analysis ảnh hưởng trực tiếp tới độ chính xác của mọi phân tích dựa trên con trỏ.

Alias – Khi hai con trỏ trỏ cùng một vùng nhớ

```c
int x = 5;
int *p = &x;
int *q = &x;   // p va q la ALIAS cua nhau (cung tro toi x)
*p = 10;
printf("%d", *q);   // In ra 10, vi p va q alias
```

Lưu ý

Aliasing khiến việc xác định ”biến nào bị thay đổi bởi lệnh nào” trở nên khó khăn cho mọi công cụ phân tích tĩnh – một nguồn gốc chính của cả false positive lẫn false negative.

Định nghĩa

Separation Logic: mở rộng logic Hoare (đã giới thiệu ở Chương 1) với toán tử ”$*$” (separating conjunction) diễn đạt hai vùng nhớ *tách biệt hoàn toàn*, không alias nhau.

Ví dụ

$\{p \mapsto x\} * \{q \mapsto y\}$ nghĩa là p trỏ tới vùng nhớ chứa x **VÀ** q trỏ tới vùng nhớ chứa y, với điều kiện 2 vùng này *không chồng lấn* – giúp suy luận cục bộ (local reasoning) về từng phần bộ nhớ độc lập.

Công cụ dùng Separation Logic: Facebook Infer

- **Infer**: công cụ static analysis công nghiệp (Meta/Facebook) dựa trên separation logic
- Có thể phân tích codebase hàng triệu dòng (Android, iOS, backend C++/Java) theo kiểu *tăng dần* (chỉ phân tích lại phần thay đổi)
- Phát hiện: null pointer dereference, memory leak, resource leak, race condition – tận dụng suy luận cục bộ để mở rộng tốt hơn nhiều so với phân tích toàn cục truyền thống

Bài tập: Mô hình hóa Double-Free bằng Memory Model

Dùng ý tưởng `valid_map` đã học, hãy mô tả (bằng lời) cách phát hiện double-free một cách hình thức.

Đáp án: Mô hình hóa Double-Free

- Trước khi thực hiện `free(p)`, kiểm tra: `valid_map[p]` phải đang là `True` (đã cấp phát, chưa free)
- Nếu `valid_map[p] = False` tại thời điểm gọi `free(p)` lần nữa $\Rightarrow$ đây chính là double-free
- Ràng buộc SMT: $\neg\text{Select}(\textit{valid_map}, p)$ tại thời điểm free thứ 2 $\Rightarrow$ nếu SAT, tồn tại tình huống double-free

Shadow Memory – Kỹ thuật của ASan (nhắc lại Chương 2)

Định nghĩa

Shadow Memory: một vùng nhớ ”bóng” song song với bộ nhớ thực, mỗi byte shadow theo dõi trạng thái (valid/invalid/redzone) của một vùng byte tương ứng trong bộ nhớ thực.

Đây chính là kỹ thuật thực tế mà AddressSanitizer (đã dùng ở Chương 2) sử dụng để phát hiện lỗi bộ nhớ khi chạy – một cài đặt *động* (dynamic) của cùng ý tưởng `valid_map` hình thức vừa học.

Redzone – Vùng đệm phát hiện Overflow

- ASan chèn thêm vùng nhớ ”redzone” (được đánh dấu invalid trong shadow memory) ngay trước và sau mỗi vùng cấp phát
- Nếu chương trình ghi/đọc vượt quá vùng cấp phát (buffer overflow), nó sẽ chạm vào redzone $\Rightarrow$ ASan phát hiện ngay lập tức
- Đây là minh họa cụ thể: mô hình hóa ”valid region” hình thức (mục 3.6) tương ứng trực tiếp với cài đặt thực tế trong công cụ đã dùng

Heap Model chi tiết hơn: Object-based thay vì Byte-based

- Thay vì coi bộ nhớ là mảng byte phẳng (mục trước), một số công cụ (vd: nhiều bounded model checker) mô hình hóa bộ nhớ theo **đối tượng** (object-based memory model)
- Mỗi `malloc` tạo ra một ”đối tượng” riêng biệt với ID duy nhất, thay vì chỉ là một dải địa chỉ trong mảng lớn
- Ưu điểm: dễ phát hiện lỗi *truy cập xuyên đối tượng* (object A ghi đè sang object B) một cách chính xác hơn mô hình byte phẳng thuần túy

Ví dụ Object-based Memory Model (khái niệm)

```
class MemoryObject:
    def __init__(self, obj_id, size):
        self.id = obj_id
        self.size = size
        self.valid = True

def check_access(obj, offset):
    if not obj.valid:
        return "USE-AFTER-FREE"
    if offset < 0 or offset >= obj.size:
        return "OUT-OF-BOUNDS"
    return "OK"
```

Mô hình này tách biệt rõ ràng từng vùng cấp phát, giúp việc kiểm tra bounds và validity đơn giản và chính xác hơn.

Bài tập: So sánh 2 mô hình bộ nhớ

So với mô hình mảng phẳng (flat array), mô hình object-based có ưu và nhược điểm gì khi kiểm chứng chương trình có *nhiều lần cấp phát/giải phóng* liên tục trong vòng lặp?

Đáp án: So sánh 2 mô hình bộ nhớ

- **Ưu điểm object-based**: mỗi lần `malloc` tạo object mới với ID riêng, dễ dàng phát hiện chính xác use-after-free/double-free cho từng object cụ thể, không nhầm lẫn giữa các lần cấp phát tại ”cùng địa chỉ” (do bộ nhớ được tái sử dụng)
- **Nhược điểm**: với vòng lặp cấp phát nhiều lần, số lượng object tăng theo số lần lặp (liên quan tới loop unwinding, mục 3.5) $\Rightarrow$ có thể làm bài toán SMT lớn hơn đáng kể so với mô hình mảng phẳng

Tổng kết 3.6

- Memory model hình thức biểu diễn bộ nhớ như một mảng địa chỉ $\rightarrow$ giá trị (Array Theory, mục 3.4)
- malloc/free được mô hình hóa qua việc theo dõi vùng ”valid”/”invalid”
- Points-to analysis và aliasing là thách thức cốt lõi của mọi phân tích liên quan con trỏ
- Separation Logic (Infer) giúp suy luận cục bộ, mở rộng tốt hơn cho codebase lớn

Từ Memory Model tới Aligned Memory Model

- Memory model cơ bản (mục 3.6) coi bộ nhớ là mảng byte phẳng – chưa xét tới **alignment** đã học ở Chương 2 (mục 2.4)
- **Aligned Memory Model**: mở rộng memory model để mã hóa *tường minh* các ràng buộc alignment, cho phép kiểm chứng tự động cả lỗi truy cập không căn chỉnh (misaligned access)

Mã hóa ràng buộc Alignment bằng SMT

```
from z3 import BitVec, Solver, URem

addr = BitVec('addr', 64)
alignment = 8   # vd: doi hoi alignment 8 byte cho kieu double

s = Solver()
s.add(URem(addr, alignment) == 0)   # Rang buoc: addr chia het cho 8
```

Toán tử URem (unsigned remainder) mã hóa chính xác điều kiện ”địa chỉ chia hết cho alignment” – nền tảng để kiểm chứng truy cập bộ nhớ có tuân thủ alignment hay không.

Kiểm tra vi phạm Alignment tự động

```
from z3 import BitVec, Solver, URem, sat

def check_misaligned_access(addr_expr, alignment):
    s = Solver()
    s.add(URem(addr_expr, alignment) != 0)  # Tim truong hop KHONG can chinh
    if s.check() == sat:
        return s.model()   # Counterexample: dia chi vi pham alignment
    return None  # Khong the vi pham (luon can chinh dung)
```

Nếu solver tìm thấy một giá trị địa chỉ thỏa mãn ”không chia hết cho alignment”, đây chính là bằng chứng hình thức về khả năng truy cập không căn chỉnh.

Kết hợp Struct Layout (Chương 2) với Aligned Memory Model

- Module `ch2_memsafe.struct_layout` (đã xây dựng ở Chương 2) tính offset của từng trường trong struct
- Aligned Memory Model dùng chính các offset này để mã hóa ràng buộc: địa chỉ trường `f` = địa chỉ struct + offset(`f`)
- Kết hợp cả hai cho phép kiểm chứng tự động: ”liệu chương trình có bao giờ truy cập một trường của struct tại địa chỉ không đúng alignment yêu cầu hay không?”

Ví dụ: Mã hóa truy cập Struct với Alignment

```
struct Point { char tag; int x; int y; };  // offset: tag=0, x=4, y=8
struct Point *p = ...;
p->x = 10;   // Dia chi truy cap: base(p) + 4
```

Với offset x = 4 (đã tính từ Chương 2), ràng buộc SMT: $(\text{base}(p) + 4) \mod 4 = 0$ – luôn đúng nếu `base(p)` tự nó đã được căn chỉnh 4-byte trở lên (do trình biên dịch đảm bảo cấp phát struct đúng alignment).

CBMC và Alignment: Kiểm tra thực tế

```
cbmc program.c --pointer-check --pointer-primitive-check
```

CBMC có thể phát hiện truy cập con trỏ không hợp lệ liên quan tới alignment khi ép kiểu con trỏ không cẩn thận (vd: ép `char*` thành `int*` tại địa chỉ không chia hết cho 4) – một lỗi thực tế phổ biến khi thao tác trực tiếp với buffer byte thô (network parsing, đã đề cập ở Chương 2).

Ví dụ lỗi Alignment thực tế qua ép kiểu con trỏ

```
unsigned char buffer[100];
// ... nhan du lieu vao buffer tu mang ...
int *value_ptr = (int *)(buffer + 1);  // Dia chi le, KHONG chia het cho 4!
int value = *value_ptr;   // Truy cap khong can chinh - UB tren mot so kien truc!
```

Lưu ý

Đây là lỗi thực tế phổ biến khi phân tích cú pháp giao thức mạng bằng cách ép kiểu con trỏ trực tiếp trên buffer byte – an toàn trên x86 (cho phép truy cập không căn chỉnh, dù chậm hơn) nhưng có thể **crash chương trình** trên ARM/SPARC (bus error).

Giải pháp an toàn: memcpy thay vì ép kiểu con trỏ

```
int value;
memcpy(&value, buffer + 1, sizeof(int));  // An toan tren MOI kien truc
```

memcpy không yêu cầu alignment của nguồn/đích, và trình biên dịch tối ưu hóa thường biến nó thành mã máy hiệu quả tương đương – đây là khuyến nghị chuẩn trong CERT C (đã học ở Chương 2) để tránh vi phạm alignment khi thao tác buffer byte thô.

Bài tập: Kiểm chứng Alignment bằng Z3

Một hệ thống cấp phát bộ nhớ cho `struct Data` (alignment yêu cầu 8 byte) bắt đầu từ địa chỉ do một bộ cấp phát tùy chỉnh (custom allocator) trả về. Hãy viết ràng buộc Z3 để tìm xem bộ cấp phát này (hàm trả về địa chỉ bất kỳ là bội số của 4, không phải 8) có thể vi phạm yêu cầu alignment hay không.

Đáp án: Kiểm chứng Alignment bằng Z3

```python
from z3 import BitVec, Solver, URem, sat

addr = BitVec('addr', 64)
s = Solver()
s.add(URem(addr, 4) == 0)    # Allocator dam bao boi so cua 4
s.add(URem(addr, 8) != 0)    # Nhung KHONG dam bao boi so cua 8
if s.check() == sat:
    print("Vi pham alignment co the xay ra:", s.model())
    # Vi du: addr = 4 (chia het cho 4 nhung khong chia het cho 8)
```

Kết quả SAT xác nhận: bộ cấp phát chỉ đảm bảo alignment 4-byte **không đủ** cho struct yêu cầu alignment 8-byte – cần sửa allocator hoặc dùng `aligned_alloc`.

Nhắc lại Padding từ Chương 2 trong ngữ cảnh hình thức

- Chương 2 (mục 2.4) đã học: trình biên dịch tự động chèn padding để đảm bảo alignment
- Trong Aligned Memory Model, padding được mã hóa như các byte ”không xác định” (unconstrained) trong vùng nhớ struct – không ảnh hưởng logic chương trình nhưng *có thể* rò rỉ nếu không `memset` (đã học ở Chương 2)
- SMT solver có thể mã hóa chính xác: byte tại offset padding có giá trị *bất kỳ* trừ khi được ghi tường minh

Kiểm chứng rò rỉ Padding bằng SMT (khái niệm)

```python
from z3 import BitVec, Solver, sat

padding_byte = BitVec('padding', 8)  # Gia tri padding truoc khi memset
s = Solver()
s.add(padding_byte != 0)  # Kiem tra: co the padding KHONG phai la 0?
if s.check() == sat:
    print("Padding co the chua du lieu ngau nhien:", s.model())
```

Nếu SAT (luôn đúng trừ khi có ràng buộc thêm từ `memset`), điều này xác nhận hình thức rằng byte padding có thể chứa dữ liệu rác – đúng như đã cảnh báo ở Chương 2.

Aligned Memory Model cho Mảng Struct

- Nhắc lại bài tập Chương 2: mảng struct cần *mỗi phần tử* đều đúng alignment, không chỉ phần tử đầu
- Ràng buộc hình thức: với mảng `Point points[n]`, địa chỉ phần tử thứ i là $\text{base} + i \times \text{sizeof(Point)}$
- Cần chứng minh: $(\text{base} + i \times \text{sizeof(Point)}) \mod \text{alignof(Point)} = 0$ với *mọi* i – đúng tự động nếu sizeof(Point) là bội số của alignof(Point) (đảm bảo bởi trình biên dịch, đã học Chương 2)

Vì sao trình biên dịch đảm bảo tính chất trên tự động

Ví dụ

Nếu `sizeof(Point) = 12` và `alignof(Point) = 4`: $12 \mod 4 = 0$, nên với mọi *i*, $(\text{base} + 12i) \mod 4 = (\text{base} \mod 4 + 12i \mod 4) \mod 4 = \text{base} \mod 4$ – không đổi so với phần tử đầu tiên. Do đó nếu phần tử đầu đúng alignment, *mọi* phần tử đều đúng.

Đây chính là lý do hình thức chứng minh được tại sao quy tắc ”kích thước struct là bội số của alignment lớn nhất” (Chương 2, mục 2.4) lại đúng đắn.

Alignment trong ngữ cảnh SIMD/Vector Instructions

- Các lệnh SIMD hiện đại (SSE, AVX trên x86) thường yêu cầu alignment nghiêm ngặt hơn (16, 32, hoặc 64 byte) so với kiểu dữ liệu cơ bản
- Truy cập không đúng alignment yêu cầu của lệnh SIMD có thể gây **crash trực tiếp** (không chỉ chậm như misaligned access thông thường)
- Aligned Memory Model mở rộng để kiểm chứng cả các yêu cầu alignment đặc biệt này khi phân tích mã tối ưu hiệu năng cao

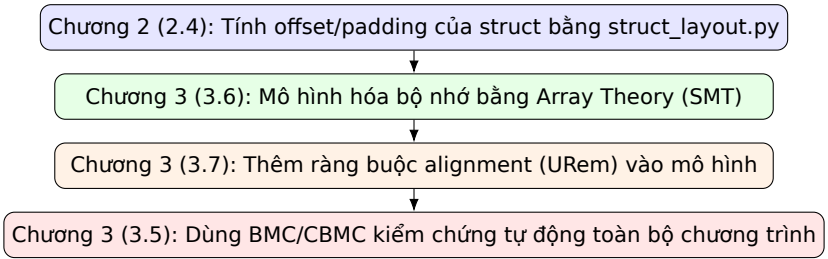
Bài tập: Chứng minh tính chất Alignment

Cho `struct S { int a, b; }` với `sizeof(S) = 8`, `alignof(S) = 4`. Nếu base chia hết cho 4, hãy chứng minh phần tử thứ 3 trong mảng `S arr[10]` (tức `arr[2]`) cũng có địa chỉ chia hết cho 4.

Đáp án: Chứng minh tính chất Alignment

- Địa chỉ `arr[2]` = base + 2×8 = base + 16
- Vì base chia hết cho 4, và 16 cũng chia hết cho 4 $\Rightarrow$ (base + 16) chia hết cho 4
- Vậy `arr[2]` luôn đúng alignment, không phụ thuộc giá trị cụ thể của base (miễn là base đã đúng alignment từ đầu)

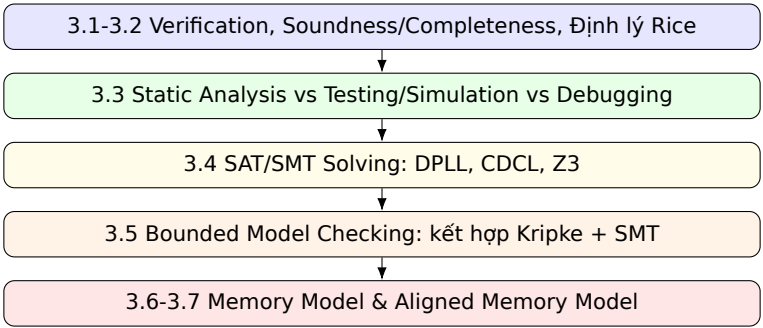
Kết hợp toàn bộ: Từ Struct Layout đến Kiểm chứng An toàn



Tổng kết 3.7

- Aligned Memory Model mở rộng memory model cơ bản với ràng buộc alignment tường minh ($addr \mod n = 0$)
- Kết hợp với struct layout (Chương 2) để kiểm chứng tự động truy cập trường đúng alignment
- Ép kiểu con trỏ trực tiếp trên buffer byte là nguồn lỗi alignment thực tế phổ biến – nên dùng `memcpy`
- CBMC có thể kiểm tra tự động các vi phạm này qua `--pointer-primitive-check`

Bản đồ tri thức Chương 3



Luyện tập trắc nghiệm – Câu 1–3

Câu 1. BMC khác Model Checking đầy đủ ở điểm nào?
A. BMC nhanh hơn luôn luôn B. BMC chỉ xét đường đi độ dài tối đa k C. BMC không dùng SAT D. Không có khác biệt
Câu 2. Completeness Threshold là gì?
A. Số biến tối đa trong SAT B. Giá trị k đủ lớn để BMC trở nên đầy đủ C. Thời gian chạy tối đa D. Số lượng solver dùng
Câu 3. Công cụ nào là Bounded Model Checker cho C?
A. Valgrind B. cppcheck C. CBMC D. gdb

Đáp án Câu 1–3

1. B (chỉ xét đường đi độ dài tối đa k).
2. B (giá trị k đủ lớn để BMC đạt đầy đủ).
3. C (CBMC).

Luyện tập trắc nghiệm – Câu 4–6

Câu 4. Array Theory trong SMT dùng để mô hình hóa gì trong Memory Model?
A. Số nguyên B. Bộ nhớ (địa chỉ $\rightarrow$ giá trị) C. Chuỗi ký tự D. Số thực
Câu 5. Toán tử nào mã hóa điều kiện alignment ”địa chỉ chia hết cho n”?
A. AND B. XOR C. URem (phần dư) D. NOT
Câu 6. Separation Logic hữu ích vì lý do gì?
A. Chạy nhanh hơn testing B. Cho phép suy luận cục bộ về vùng nhớ tách biệt C. Không cần SMT solver D. Chỉ dùng cho Java

Đáp án Câu 4–6

4. B (Bộ nhớ: địa chỉ $\rightarrow$ giá trị).
5. C (URem – phần dư của phép chia).
6. B (Suy luận cục bộ về vùng nhớ tách biệt, giúp mở rộng tốt hơn).

Bảng thuật ngữ cốt lõi Chương 3 (1/2)

Verification/Validation	Đúng theo đặc tả / Đúng theo nhu cầu người dùng
Soundness/Completeness	Không bỏ sót lỗi / Không báo sai lỗi
Định lý Rice	Không thể quyết định tổng quát mọi thuộc tính ngữ nghĩa không tầm thường
SAT/SMT	Bài toán thỏa mãn công thức mệnh đề / mở rộng với các lý thuyết (số học, mảng)
DPLL/CDCL	Thuật toán nền tảng và cải tiến hiện đại của SAT solver

Bảng thuật ngữ cốt lõi Chương 3 (2/2)

BMC	Kiểm tra vi phạm trong phạm vi k bước bằng mã hóa SAT/SMT
Completeness Threshold	Giá trị k đủ lớn để BMC trở nên đầy đủ
CBMC	Bounded Model Checker công nghiệp cho C
Memory Model	Biểu diễn hình thức bộ nhớ (địa chỉ $\rightarrow$ giá trị)
Aligned Memory Model	Memory Model có ràng buộc alignment tường minh

Câu hỏi ôn tập tổng hợp (1/3)

1. Vì sao BMC không thể chứng minh ”an toàn mãi mãi” nếu không đạt completeness threshold?
2. CBMC dùng `--unwind` để làm gì? Điều gì xảy ra nếu giá trị này quá nhỏ?
3. Points-to analysis và Aliasing liên quan gì tới độ chính xác của static analysis?

Câu hỏi ôn tập tổng hợp (2/3)

4. Separation Logic giải quyết vấn đề gì mà logic Hoare thông thường gặp khó khăn?
5. Toán tử URem dùng để mã hóa ràng buộc gì trong Aligned Memory Model?
6. Vì sao ép kiểu con trỏ trực tiếp trên buffer byte có thể gây lỗi alignment?

Câu hỏi ôn tập tổng hợp (3/3)

7. K-Induction khác BMC thông thường ở điểm nào?
8. Object-based memory model có ưu điểm gì so với flat array model?
9. Shadow memory (ASan) liên hệ thế nào với valid_map hình thức đã học?

Đáp án gợi ý câu 1–3

1. Vì BMC chỉ xét đường đi $\leq k$ bước – vi phạm có thể tồn tại ở bước $k+1$ trở đi nếu k chưa đạt completeness threshold
2. `--unwind` xác định số lần mở vòng lặp tối đa; nếu quá nhỏ, CBMC báo unwinding assertion failure và kết quả không đáng tin cậy cho các trường hợp cần nhiều vòng lặp hơn
3. Points-to analysis xác định con trỏ có thể trỏ tới đâu; alias (nhiều con trỏ cùng trỏ một vùng) làm việc này khó chính xác, ảnh hưởng trực tiếp độ tin cậy của mọi phân tích dựa trên con trỏ

Đáp án gợi ý câu 4–6

4. Separation Logic cho phép suy luận *cục bộ* về từng vùng nhớ tách biệt (không alias), giúp mở rộng phân tích cho codebase lớn tốt hơn logic Hoare truyền thống phải xét toàn bộ trạng thái bộ nhớ
5. URem mã hóa ”địa chỉ mod alignment = 0” – điều kiện địa chỉ chia hết cho yêu cầu alignment
6. Vì địa chỉ sau khi cộng offset lẻ có thể không còn chia hết cho alignment yêu cầu của kiểu đích, gây truy cập không căn chỉnh (misaligned access), có thể là UB hoặc crash trên một số kiến trúc

Đáp án gợi ý câu 7–9

7. K-Induction chứng minh bằng quy nạp (base case + inductive step), có thể suy ra an toàn với *mọi* số bước mà không cần tính completeness threshold tường minh như BMC thuần túy
8. Object-based dễ phát hiện chính xác lỗi truy cập xuyên đối tượng và use-after-free/double-free cho từng object riêng biệt, dù có thể tốn kém hơn với nhiều lần cấp phát
9. Shadow memory là cài đặt *động, thực tế* của cùng ý tưởng valid_map hình thức: một vùng ”theo dõi” song song đánh dấu trạng thái hợp lệ của bộ nhớ chính

Luyện tập trắc nghiệm bổ sung – Câu 7–9

Câu 7. Kỹ thuật nào giúp giảm số interleaving cần xét khi BMC hệ đa luồng?
A. Loop unwinding B. Partial Order Reduction C. CDCL D. Shadow memory
Câu 8. Công cụ Infer của Meta/Facebook dựa trên nền tảng lý thuyết nào?
A. Model Checking thuần túy B. Separation Logic C. Fuzzing D. Chỉ dùng testing
Câu 9. Redzone trong ASan có vai trò gì?
A. Tăng tốc chương trình B. Phát hiện truy cập vượt vùng cấp phát C. Nén dữ liệu D. Mã hóa dữ liệu

Đáp án Câu 7–9

7. B (Partial Order Reduction).
8. B (Separation Logic).
9. B (Phát hiện truy cập vượt vùng cấp phát – buffer overflow).

Bảng công thức cần nhớ Chương 3

Khái niệm	**Công thức/Ký hiệu**
BMC formula	$I(s_0) \wedge \bigwedge R(s_i, s_{i+1}) \wedge \bigvee \text{bad}(s_i)$
Alignment check	$addr \mod n = 0$ (dùng URem)
CNF	Hội của các mệnh đề tuyển (AND của OR)
Sound	Không có false negative
Complete	Không có false positive

Bài tập tổng hợp lớn (chuẩn bị kiểm tra)

Cho một hàm C phân tích gói tin mạng, dùng vòng lặp đọc `header_len` byte vào buffer 256 byte cố định. Hãy mô tả quy trình đầy đủ dùng các kỹ thuật đã học trong Chương 3 để kiểm chứng hàm này không có buffer overflow.

Gợi ý đáp án bài tập tổng hợp

1. Chạy **CBMC** với `--bounds-check --unwind N` (N đủ lớn hơn giá trị lớn nhất có thể của `header_len`)
2. Nếu CBMC báo FAILURE, lấy counterexample (giá trị `header_len` cụ thể) để xác nhận và sửa mã nguồn
3. Nếu CBMC báo SUCCESS *và* không có unwinding assertion failure $\Rightarrow$ đã đạt completeness threshold cho vòng lặp này, kết quả đáng tin cậy
4. Bổ sung kiểm tra alignment nếu hàm có ép kiểu con trỏ trên buffer (mục 3.7)

Cập nhật dự án Python: module ch3_staticsat

Module `ch3_staticsat` trong `softsec-toolkit` gồm:

- `sat_solver.py`: cài đặt SAT solver mini theo thuật toán DPLL (unit propagation + backtracking)
- `smt_wrapper.py`: các hàm tiện ích bọc quanh thư viện `z3-solver` cho các bài toán thường gặp (kiểm tra tràn số, alignment, sinh test case)
- `bounded_checker.py`: Bounded Model Checker mini, áp dụng trực tiếp lên `KripkeStructure` đã xây dựng ở Chương 1

Demo module: bounded_checker

```python
from softsec.ch1_models.kripke import build_login_example
from softsec.ch3_staticsat import bmc_check

model = build_login_example()
result = bmc_check(model, bad_states={"locked"}, k=3)
print(result.violated, result.counterexample)
# True ['idle', 'locked']
```

Module này áp dụng *trực tiếp* lý thuyết BMC vừa học lên chính mô hình Kripke đã xây dựng từ Chương 1 – khép kín vòng tròn kiến thức xuyên suốt học phần.

Checklist tự đánh giá cuối Chương 3

- ☐ Tôi hiểu sự khác biệt giữa BMC và model checking đầy đủ, và ý nghĩa của completeness threshold
- ☐ Tôi có thể sử dụng CBMC (khái niệm) để kiểm tra buffer overflow trên một hàm C đơn giản
- ☐ Tôi hiểu cách memory model biểu diễn malloc/free/alignment bằng SMT
- ☐ Tôi có thể viết ràng buộc Z3 cơ bản cho kiểm tra alignment

Danh sách kỹ năng thực hành đã đạt được (Chương 3)

Sau Chương 3, sinh viên đã có thể:

- Viết công thức BMC đơn giản bằng tay và hiểu ý nghĩa completeness threshold
- Sử dụng CBMC (khái niệm) để kiểm tra buffer overflow, division by zero trên mã C
- Viết ràng buộc Z3 cho memory model cơ bản (valid/invalid region) và alignment (URem)
- Phân biệt rõ các loại memory model (flat array vs object-based) và ưu nhược điểm

So sánh nhanh: Chương 2 vs Chương 3

	Chương 2	**Chương 3**
Công cụ	cppcheck, clang-tidy, Valgrind (thực hành)	Z3, CBMC (lý thuyết + thực hành)
Độ chính xác	Không đảm bảo sound/complete rõ ràng	Có thể sound trong phạm vi k
Nền tảng	Pattern/heuristic	Toán học hình thức (SAT/SMT)

Thảo luận: Vì sao ngành công nghiệp chưa dùng Formal Verification rộng rãi

- Chi phí đào tạo nhân lực hiểu được logic hình thức, SMT solver còn cao
- Với codebase legacy khổng lồ, việc xây dựng mô hình hình thức tốn nhiều công sức
- Đây là lý do các công cụ như CBMC cố gắng *tự động hóa* tối đa việc trích xuất mô hình từ mã nguồn thực, giảm rào cản áp dụng

Điểm cần lưu ý khi ôn tập Chương 3

- Phân biệt rõ Sound/Complete và nhớ ví dụ cụ thể cho mỗi công cụ đã học xuyên suốt cả 3 chương
- Luyện giải SAT bằng tay và viết ràng buộc Z3 cơ bản – đây là kỹ năng thực hành quan trọng nhất
- Ôn lại toàn bộ 6 câu trắc nghiệm có đáp án ở cả 2 phần của Chương 3 trước khi thi giữa kỳ

Hỏi đáp

Câu hỏi?

Chuẩn bị cho Chương 4

Chương 4 – Phát hiện lỗ hổng phần mềm: Phân tích động sẽ chuyển hướng từ kiểm chứng hình thức (static, Chương 3) sang kiểm thử động (dynamic):

- 4.1–4.2: Giới thiệu kiểm thử bảo mật, code coverage (nhắc lại và mở rộng từ Chương 1)
- 4.3–4.4: Theo dõi thực thi chương trình, sinh đường thực thi
- 4.5–4.6: Fuzzing hộp đen (black-box) và hộp trắng (white-box, dùng chính kỹ thuật symbolic execution + SMT đã học ở Chương 3!)

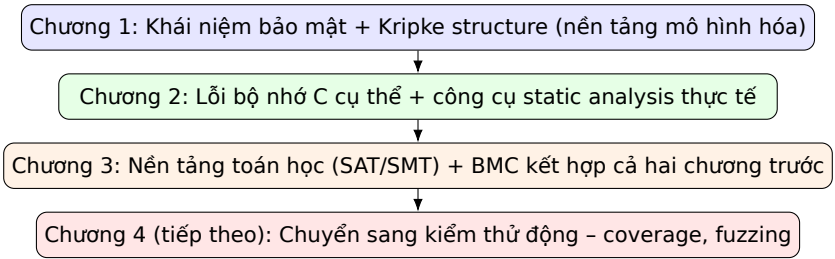
Vai trò Chương 3 trong bài tập lớn

Nếu bài tập lớn liên quan tới kiểm chứng hình thức, báo cáo nên trình bày: (1) mô hình Kripke hoặc memory model đã xây dựng, (2) thuộc tính cần kiểm chứng viết bằng LTL/CTL hoặc ràng buộc SMT, (3) kết quả chạy CBMC/Z3 kèm counterexample nếu có, (4) phân tích completeness threshold đã đạt được hay chưa.

Nguồn thực hành thêm

- CBMC GitHub repository – tài liệu và ví dụ chính thức
- Z3 Playground (rise4fun) – thử nghiệm ràng buộc SMT trực tuyến
- ”Decision Procedures” (Kroening, Strichman) – sách chuyên sâu về SAT/SMT và ứng dụng kiểm chứng
- SV-COMP (Software Verification Competition) – benchmark chuẩn để so sánh các bounded model checker

Kết nối tổng thể: Ba chương đã học



Kết thúc Chương 3

Hoàn thành Chương 3: Phát hiện lỗ hổng phần mềm – Phân tích tĩnh

Hẹn gặp lại ở Chương 4: Phát hiện lỗ hổng phần mềm – Phân tích động

Câu hỏi thảo luận mở cuối chương

Nếu bạn phải chọn *một* kỹ thuật duy nhất trong toàn bộ Chương 3 (BMC, memory model, hay SAT/SMT solving) để học sâu hơn cho sự nghiệp về an toàn phần mềm, bạn sẽ chọn gì và tại sao?

Gợi ý thảo luận

- Không có câu trả lời ”đúng” duy nhất – điều quan trọng là hiểu *mối liên hệ* giữa cả ba: SAT/SMT là công cụ toán học nền tảng, memory model là cách áp dụng nó vào bài toán bộ nhớ cụ thể, BMC là kỹ thuật kết hợp cả hai với Kripke structure để kiểm chứng tự động
- Trong thực tế công nghiệp, hiểu biết về SMT solving (Z3) là kỹ năng có thể ứng dụng rộng nhất, vượt ra ngoài phạm vi an toàn phần mềm (tối ưu hóa, lập lịch, xác minh phần cứng...)

Tự đánh giá cuối cùng trước khi thi giữa kỳ

- Ôn lại toàn bộ 4 phần đã học: Chương 1 (2 phần), Chương 2 (2 phần), và giờ là Chương 3 (2 phần)
- Thực hành lại các bài tập tính toán: risk score, cyclomatic complexity, struct alignment, giải SAT bằng tay
- Đảm bảo phân biệt rõ ràng các cặp khái niệm dễ nhầm: Verification/Validation, Sound/Complete, Static/Dynamic, SAT/SMT

Chuyển tiếp

Tiếp theo: Chương 4 – Phát hiện lỗ hổng phần mềm: Phân tích động